



Aula 03 — Interfaces em Java

Professor: Júlio Oliveira



SENAC — CURSO JAVA DEVELOPER ESSENTIALS

O que veremos hoje

Nesta aula, mergulhamos em um dos conceitos mais poderosos da Programação Orientada a Objetos em Java: as **Interfaces**. Confira a agenda completa:

1

O que é uma interface

Conceito e analogia do mundo real

2

Declarando e implementando

Palavras-chave `interface` e `implements`

3

Interface x Classe Abstrata

Diferenças e quando usar cada uma

4

Múltiplas interfaces

Default methods — Java 8+

5

Polimorfismo via interface

Boas práticas e código desacoplado

6

Atividade prática

Sistema de meios de pagamento

Conceito + Analogia

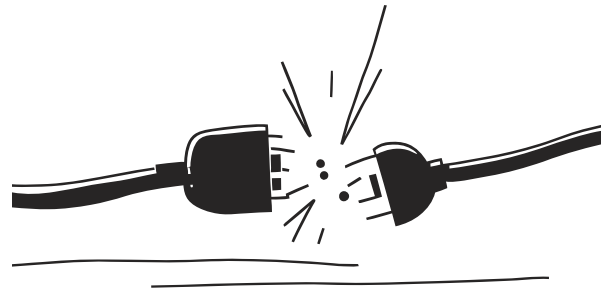
O que é uma interface?

Uma interface é um **contrato** que define **O QUÊ** uma classe deve fazer, sem dizer **COMO**. É como uma tomada elétrica: define o formato dos pinos, e qualquer aparelho que respeite esse contrato pode se conectar.

Definição técnica

- Coleção de métodos abstratos (sem corpo)
- Não pode ser instanciada diretamente
- Sem atributos de instância — apenas constantes
- Define contrato entre chamador e implementador

Analogia da tomada



- **Tomada** = interface (define o formato dos pinos)
- **TV, Geladeira, Carregador** = classes que implementam
- Cada um respeita o contrato à **sua maneira**

Sintaxe Java

Declarando e implementando uma interface

Usa-se a palavra-chave `interface` para declarar e `implements` para que uma classe assuma o contrato. A classe é **obrigada** a fornecer o corpo de todos os métodos.

```
public interface Veiculo {
    int RODAS_MIN = 2;    // constante (public static final)
    void acelerar();       // métodos abstratos (public abstract)
    void frear();
    String getModelo();
}

public class Carro implements Veiculo {
    private String modelo;
    public Carro(String modelo) { this.modelo = modelo; }

    @Override public void acelerar() { System.out.println(modelo + " acelerando..."); }
    @Override public void frear()   { System.out.println(modelo + " freando."); }
    @Override public String getModelo() { return modelo; }
}
```

 A classe `Carro` assina o contrato `Veiculo` e deve implementar **todos** os métodos declarados na interface — caso contrário, o compilador acusará erro.

Comparativo

Interface x Classe Abstrata

Ambas servem para abstração, mas têm propósitos diferentes. Use **classe abstrata** quando há código comum a compartilhar; use **interface** para definir um contrato.

Característica	Interface	Classe Abstrata
Herança	Múltipla (implements várias)	Simples (extends uma)
Métodos com corpo	Apenas default/static	Sim, livremente
Atributos	Apenas constantes	Atributos normais
Construtor	Não tem	Pode ter
Quando usar	Contrato / capacidade	Compartilhar código + abstrair

✔ **Use interface** quando classes muito diferentes precisam compartilhar um mesmo contrato de comportamento.

ℹ **Use classe abstrata** quando há lógica comum que pode ser herdada e reutilizada pelas subclasses.

Java 8+

Múltiplas interfaces e default methods

Java não permite herança múltipla de classes, mas uma classe pode **implementar várias interfaces**. Desde o Java 8, interfaces podem ter métodos com implementação padrão (default).

```
public interface Nadador {  
    void nadar();  
    default void respirarFora() {  
        System.out.println("Respirando ar.");  
    }  
}  
  
public interface Voador {  
    void voar();  
    static String tipo() {  
        return "Animal aéreo";  
    }  
}  
  
public class PatoSelvagem  
    implements Nadador, Voador {  
    @Override  
    public void nadar() {  
        System.out.println("Nadando...");  
    }  
    @Override  
    public void voar() {  
        System.out.println("Voando...");  
    }  
}
```

Pontos-chave

default

Método com implementação padrão — pode ser sobrescrito pela classe implementadora.

static

Método utilitário da interface — chamado via `NomeInterface.metodo()`, não pode ser sobrescrito.

Múltiplas interfaces

`PatoSelvagem` é ao mesmo tempo `Nadador` e `Voador` — flexibilidade que herança simples não oferece.

Poder das Interfaces

```
List<Veiculo> garagem = new ArrayList<>();
garagem.add(new Carro("Fusca"));
garagem.add(new Moto("CG 160"));
garagem.add(new Caminhao("FH 540"));

for (Veiculo v : garagem) {
    v.acelerar(); // cada um acelera à sua maneira
}
```

💡 **Destaque:** A variável é do tipo `Veiculo` (interface), mas o comportamento real é da classe concreta. O compilador garante o contrato; o polimorfismo garante a flexibilidade.

Carro

"Fusca acelerando..."

Moto

"CG 160 acelerando..."

Caminhão

"FH 540 acelerando..."

Recomendações

Boas práticas com interfaces

Recomendações para usar interfaces de forma eficaz e idiomática em Java.



Programe para a interface

Prefira `List` em vez de `ArrayList` nas declarações — isso desacopla o código da implementação concreta.



Nomeie por capacidade ou papel

Use nomes como `Comparable`, `Runnable`, `Veiculo`, `Pagamento` — deixe claro o que a interface representa.



Sempre use `@Override`

Ao implementar métodos da interface, use sempre a anotação `@Override` — o compilador valida e o código fica mais legível.



Interfaces pequenas e coesas

Siga o princípio ISP (Interface Segregation): interfaces devem ter responsabilidade única e não forçar implementações desnecessárias.



Default methods com parcimônia

Use `default` para evoluir interfaces sem quebrar código existente — mas não transforme a interface em uma classe disfarçada.



Atividade Prática — Meios de Pagamento

Vamos praticar criando um sistema simples de meios de pagamento. O objetivo é ver na prática como uma mesma interface pode ser implementada por classes muito diferentes.

Enunciado

01

Crie a interface `MeioPagamento` com os métodos: `void pagar(double valor)` e `String tipo()`

02

Crie **3 classes** implementando a interface: `CartaoCredito`, `Pix` e `Boleto`

03

Cada classe deve imprimir uma mensagem coerente com seu tipo ao chamar `pagar()`

04

No `main`, crie uma `List<MeioPagamento>`, adicione um objeto de cada e itere chamando `pagar(100.0)`

Saída esperada

Pagando R\$ 100.0 com Cartão de Crédito.

Pagando R\$ 100.0 via Pix. Chave: ...

Gerando Boleto de R\$ 100.0...



Tempo estimado: 30 minutos

Entrega: arquivo `.java` compilando e executando sem erros.



Dica: Lembre-se de usar `@Override` em todos os métodos implementados e de declarar a variável como `List<MeioPagamento>` — não como `ArrayList`!

Recapitulando + Referências

Nesta aula exploramos um dos pilares mais poderosos da OOP em Java — as interfaces. Veja os pontos essenciais:

Recapitulando

- **Interface = contrato** — define O QUÊ a classe faz, não COMO
- Declarada com `interface`, implementada com `implements`
- Permite implementar **várias interfaces** — flexibilidade superior à herança múltipla
- **Default methods** (Java 8+) evoluem interfaces sem quebrar código existente
- Viabilizam **polimorfismo** e código desacoplado e extensível

Referências Bibliográficas

DEITEL, P.; DEITEL, H. Java: Como Programar. 10ª ed. Pearson, 2017.

BLOCH, J. Effective Java. 3ª ed. Addison-Wesley, 2018.

SIERRA, K.; BATES, B. Use a Cabeça! Java. 2ª ed. Alta Books, 2010.

Casa do Código — Desbravando Java e Orientação a Objetos.

Oracle Docs: docs.oracle.com/javase/tutorial/java/land/

  **Próxima aula:** Módulo 07 — Aula 04. Fique ligado nas novidades do curso!